

NEXUS AI

N08 — System Architecture Document

Complete Product Architecture • Local-First • Multi-Model • RAG • Agentic • Tool-Controlled

Version: 0.1 **Status:** Architecture Foundation **Scope:** MVP → Future

Architecture objective: Define the complete logical architecture of NEXUS AI before implementation. NEXUS is an offline-first local AI workstation that orchestrates multiple local models, knowledge retrieval, persistent memory and controlled tools through a unified application.

USER → EXPERIENCE → ORCHESTRATION → INTELLIGENCE / KNOWLEDGE / TOOLS → LOCAL RUNTIME → STORAGE

1. Architecture Vision

NEXUS AI is a local-first AI platform rather than a single model application. The system separates user experience, orchestration, model inference, retrieval, memory, tools and local runtime so that each capability can evolve independently.

The architecture is model-agnostic. Ollama is the initial local inference adapter because the current NEXUS environment already contains local models through Ollama. The application must not hard-code business logic directly into one model.

Primary Architectural Goals

- Run core AI workflows locally without a cloud dependency.
- Orchestrate multiple specialist models through one interface.
- Provide grounded RAG answers with traceable sources.
- Allow agents to use tools through explicit permissions.
- Support coding workflows inside controlled project workspaces.
- Preserve conversation, project and knowledge memory.
- Make model/tool failures recoverable and observable.
- Allow future online services without redesigning the local core.

2. Architecture Principles

Principle	Architecture Rule
Local-first	Core chat, RAG, memory and approved tools operate locally.
Model-agnostic	LLMs are accessed through provider/model adapters.
Orchestrator-owned control	Models request actions; NEXUS decides whether and how actions execute.
Least privilege	Tools receive only the permissions required for the current task.
Evidence-aware	Knowledge answers retain source metadata and retrieval context.
Observable	Model calls, routing, retrieval, tool calls and failures are measurable.
Replaceable components	Vector store, embedding model, model provider and UI layers remain replaceable.
Human control	Destructive or high-impact actions require confirmation or policy approval.
Offline degradation	Loss of internet must not break the local core.

3. Logical System Architecture

NEXUS is divided into seven major layers. Each layer has a narrow responsibility and communicates through explicit internal contracts.

Layer	Major Components	Responsibility
Experience	Desktop/Web UI, Chat, Workspace, Model Center, Knowledge Center	User interaction and presentation.
Application API	API gateway, sessions, request validation, streaming	Application boundary and client communication.
Orchestration	Router, planner, context manager, agent runtime, verifier	Decides what should happen and coordinates execution.
Intelligence	Ollama adapter, model registry, embeddings, reranker	Inference and AI-specific operations.
Knowledge & Memory	Document pipeline, vector store, metadata store, memory service	Retrieval, citations and persistent context.
Tools	File, search, terminal, code/test and future connectors	Controlled external actions.
Runtime & Storage	Local DB, filesystem, model runtime, cache, logs	Persistence and machine-local execution.

Canonical Request Flow

User Request → API → Session/Policy → Router → Context Builder → Planner/Direct Model → RAG/Tools as needed → Verifier → Response Stream → Conversation/Telemetry Persistence.

4. High-Level Component Map

The following component boundaries are normative for the initial implementation:

Component	Consumes	Produces
UI Client	User input, server events	API requests, tool approvals, rendered output
API Layer	UI requests	Validated application commands/events
Session & Policy	Identity, project, permissions	Allowed capability context
Model Router	Request + context + model registry	Selected model/strategy
Agent Runtime	Plan + tools + model adapters	Task state, tool calls, intermediate results
Context Manager	Conversation, project, memory, retrieved chunks	Bounded prompt/context package
RAG Engine	Documents, query	Ranked evidence chunks + citations
Tool Gateway	Tool request + policy	Validated tool result
Model Gateway	Normalized model request	Streaming/non-streaming model output
Verifier	Draft + evidence + task state	Quality/grounding decision
Persistence	Domain objects/events	Stored state
Observability	Runtime events	Logs, metrics, traces

5. Runtime Architecture

The runtime is designed around a local application process and explicit adapters. The first production-like desktop deployment can package the UI and local application services together while keeping model inference behind a provider interface.

Runtime Area	Initial Choice	Future Extension
LLM Runtime	Ollama	Additional local runtimes/providers
Application Backend	Local API/service	Remote/private server mode
Database	Local relational database	PostgreSQL for multi-user deployment
Vector Store	Local vector-capable store	Dedicated vector database
File Storage	User-controlled local workspace	Object storage adapter
Embeddings	Local embedding model	Provider abstraction
Reranking	Local/optional reranker	Specialized reranking service
UI	Desktop-oriented web UI	Multi-device client
Packaging	Windows-first local package	Cross-platform packaging

6. Model Architecture

NEXUS does not treat installed models as interchangeable strings. Each model is represented in a registry with capabilities, context constraints, resource requirements, health status and routing metadata.

Model Metadata	Examples
Identity	provider, model name, version/tag
Capabilities	general, reasoning, coding, summarization, tool-use
Context	maximum context, preferred context, output limits
Resources	estimated RAM/VRAM, load state, concurrency
Performance	latency, tokens/sec, benchmark scores
Reliability	availability, error rate, fallback priority

Model Metadata	Examples
Policy	allowed tasks, privacy class, tool capability

Initial local model pool: Qwen2.5-Coder 7B, DeepSeek-R1 8B, Qwen2.5 3B, Llama 3.1 8B and the alternative Qwen coder model currently installed. These are starting candidates, not permanent role assignments; NEXUS should benchmark them.

Model Routing

Request → classify task → inspect available models → score candidates → select model → execute → detect failure → fallback if permitted → verify output.

Routing must account for task type, model capability, context size, current resource pressure, latency target and benchmark performance.

7. Agentic Architecture

Agentic workflows are coordinated by an Agent Runtime. A model cannot directly execute arbitrary operating-system actions. It produces structured tool requests that pass through policy checks.

Stage	Function
Intent	Understand the user's goal and constraints.
Planning	Create a bounded task plan when multiple steps are required.
Context	Assemble conversation, project, memory and retrieved evidence.
Execution	Call model and approved tools through gateways.
Observation	Capture tool/model results and update task state.
Verification	Check completion, evidence, tests or constraints.
Completion	Return result, summary and relevant evidence.
Recovery	Retry, change strategy or ask for user input when blocked.

8. RAG Architecture

RAG is a subsystem available to the orchestrator, not an unconditional step for every request.

Document → Parse → Normalize → Chunk → Metadata → Embed → Index → Retrieve → Hybrid/Rerank → Context Builder → Generate → Citation Mapping → Verify.

Subsystem	Responsibility
Ingestion	Accept local documents and supported project files.
Parsing	Extract text, metadata and where possible structure.
Chunking	Create retrieval units while preserving source references.
Embedding	Generate local vector representations.
Retrieval	Find candidate evidence using semantic and/or lexical search.
Reranking	Improve relevance of top candidates.
Citation	Preserve document/page/chunk provenance.
Grounding	Restrict answer context to approved evidence when required.

9. Memory Architecture

Memory Layer	Scope	Purpose
Conversation	Single conversation	Recent dialogue and active context.
Session	Current task/session	Temporary task state and tool results.
Project	One workspace	Files, decisions, tasks and project-specific context.
Knowledge	Indexed sources	Reusable document knowledge with provenance.

Memory Layer	Scope	Purpose
User	Explicitly approved preferences	Long-term personalization with user controls.

Memory retrieval must be selective. NEXUS should not dump all stored memory into every prompt. Relevance, privacy scope and context budget determine what is included.

10. Tool Architecture

Tools are registered capabilities with schemas, permissions and execution adapters.

Tool	Initial Purpose	Risk Level
File Search	Find files in approved workspace	Low
File Read	Read approved files	Low
File Write/Edit	Modify approved workspace files	Medium
Terminal	Run approved commands	High
Test Runner	Execute project tests	Medium
Git	Inspect status/diff; later controlled write operations	Medium
Knowledge Search	Search local RAG knowledge	Low
System Info	Read selected local resource information	Low

Every tool call passes through: schema validation → permission policy → scope validation → execution → result sanitization → audit event.

11. Data & Event Architecture

NEXUS should use domain events internally so long-running agent workflows and UI streaming do not depend on fragile direct component coupling.

Event	Example Payload
chat.message.created	conversation_id, message_id
model.request.started	model, task_id, timestamp
model.response.delta	task_id, text_delta
retrieval.completed	query_id, sources, latency
tool.requested	tool, arguments, approval_state
tool.completed	tool, result_summary, duration
agent.step.completed	task_id, step_id, status
verification.completed	task_id, score/status
task.completed	task_id, result
task.failed	task_id, error_code, recovery_state

Events support UI streaming, telemetry, debugging and future background workers. They are not automatically equivalent to durable business records; persistence rules are defined per event type.

12. End-to-End Interaction Flows

12.1 Normal Chat

User → UI → API → Session/Policy → Router → Model Gateway → Ollama → Streaming Response → Conversation Store → UI.

12.2 Automatic Model Routing

User → Router → classify → model registry → capability/resource scoring → selected model → model gateway → response → verifier if required.

12.3 RAG Question

User → Router → RAG decision → query rewrite if required → retrieval → rerank → context builder → selected LLM → citation mapping → verification → response.

12.4 Coding Agent

User → Planner → workspace inspection → file tools → coding model → edit → test runner → observe failure/success → repair loop → verifier → final diff/summary.

12.5 Tool Approval

Agent requests tool → policy evaluates → if approval required, UI asks user → approved request executes → result returned to agent → audit event recorded.

13. Security Architecture Boundary

Boundary	Control
UI → API	Authenticated local session and request validation.
API → Orchestrator	Task identity, project scope and policy context.
Orchestrator → Model	Prompt/context filtering and model policy.
Orchestrator → Tool Gateway	Allowlist, schema validation and permission checks.
Tool → Filesystem	Workspace sandbox and path validation.
Tool → Terminal	Command policy, working directory restrictions and confirmation.
RAG → Documents	Collection permissions and source provenance.
Logs	Secret redaction and controlled retention.
External network	Disabled by default for offline core; explicit connector policy later.

Detailed sandboxing and permission rules belong to N16. This architecture only establishes the security boundary and responsibility split.

14. Failure & Recovery Architecture

Failure	Expected Response
Model unavailable	Mark unhealthy, try approved fallback or request user choice.
Model timeout	Cancel/timeout task, preserve state and offer retry.
RAG failure	Return transparent retrieval error or continue only if policy allows ungrounded response.
Tool failure	Capture structured error, allow bounded retry or ask user.
Database failure	Fail safely, preserve unsaved task state where possible.
Agent loop failure	Stop after bounded retries and expose reason.
Resource pressure	Reduce concurrency, unload models or ask user to switch model.
Application restart	Recover durable conversations/tasks according to persistence policy.

15. Performance Architecture

NEXUS is resource-constrained by the host device. The architecture therefore treats model loading, context size, concurrency and GPU/RAM pressure as first-class runtime signals.

Metric	Purpose
Time to first token	Perceived chat responsiveness.
Tokens/second	Inference throughput.
Model load time	Model switching cost.

Metric	Purpose
Context size	Memory and latency control.
RAG retrieval latency	Knowledge response performance.
Tool execution duration	Agent workflow performance.
End-to-end task latency	Overall user experience.
RAM/VRAM usage	Prevent system instability.
Model error rate	Routing reliability.

16. Deployment Topology

MVP deployment is local-first:

NEXUS UI + Application Services + Local Database + Vector Store + Ollama + Local Workspace + Logs → User Device.

Optional future topology:

Client UI → NEXUS API → Local/Private Orchestrator → Model Providers + Vector Store + Storage + Optional Cloud Connectors.

The local topology remains the reference architecture for privacy and offline operation.

17. Extensibility Strategy

Extension	Required Abstraction
New LLM provider	Model Provider Adapter
New local runtime	Inference Runtime Adapter
New embedding model	Embedding Provider Interface
New vector database	Vector Store Interface
New tool	Tool Registry + Tool Adapter
New UI client	API/Event Contract
Cloud mode	Remote Service Adapter
Multimodal model	Capability-aware Model Adapter

18. Architecture Decision Summary

Decision	Status	Reason
Local-first core	Approved	Privacy, offline capability and user control.
Ollama as initial local runtime	Approved	Matches existing local model environment and provides a provider boundary.
Model-agnostic orchestration	Approved	Avoid lock-in and enable specialist routing.
Agent runtime separate from model gateway	Approved	Models generate reasoning; orchestration controls execution.
Tools behind gateway	Approved	Required for permissions, validation and auditing.
RAG as optional orchestrated capability	Approved	Not every request requires retrieval.
Memory separated by scope	Approved	Prevents context pollution and improves privacy.
Desktop/local-first MVP	Approved	Primary use case is personal local AI workstation.
Cloud connectors optional	Future	Extend capability without making cloud mandatory.

19. MVP Architecture Boundary

The MVP must implement the minimum complete loop:

Local UI → API → Session/Policy → Model Router → Model Gateway → Ollama → Streaming Chat → Conversation Persistence.

Then add the second proof loop:

Local Document → Ingestion → Embedding/Index → Retrieval → Context → Local Model → Citation → Response.

The first agent proof loop is:

User Task → Planner → Approved File Tool → Coding Model → Test Tool → Verification → Result.

These three loops establish the architecture before advanced multi-agent autonomy is introduced.

20. Architecture Quality Gates

- No direct UI dependency on a specific LLM.
- No agent can bypass the Tool Gateway.
- No tool can escape its configured scope.
- RAG answers preserve provenance.
- Model adapters expose normalized interfaces.
- Long-running tasks have cancellation and bounded retry behavior.
- Core workflows remain functional without internet.
- Runtime telemetry can explain model, retrieval and tool activity.
- Persistence boundaries are explicit.
- New providers/tools can be added without rewriting orchestration.

21. Architecture Roadmap Dependencies

N07 defines technical requirements and constraints. N08 establishes this system architecture. N09 should convert these logical boundaries into concrete internal components and interfaces. N10 should formalize data movement. N11–N16 then detail the AI runtime, agent orchestration, tools, RAG, memory and security layers.

Next document: N09 — Component Architecture Document.

22. Final Architecture Statement

NEXUS AI is architected as a local-first, model-agnostic, agent-capable AI workstation. The central design decision is to place an orchestration and policy layer between the user and local models. RAG, memory and tools become controlled capabilities rather than features directly embedded inside a single chatbot. This makes the system extensible from local chat to coding agents, research agents and multi-agent workflows while preserving privacy, observability and user control.

Document: N08 — System Architecture Document • **Version:** 0.1 • **Status:** Architecture Foundation